跳至内容

历史聊天记录

新聊天
CtrlShiftO

搜索聊天
CtrlK

图片

应用

深度研究

Codex

GPT

项目

新项目

主线 2

最近

01-软件 调研

02-CaseSpec 数据结构设计

Ini2----自动化 CAD/CAE 项目 提示词

Ini----项目管理对话结构

01 自动化 CAD/CAE 调研

铺层库

你的聊天

打不开 Word 文件解决办法

Cursor 环境配置

Codex 与 Cursor 介绍

Edge 浏览器收藏夹问题

聊天记录与卡住问题

Phạm Thị Ngọc Trinh
Plus

ChatGPT Auto
分享

你说：

项目名称：自动化 CAD/CAE 这是 Research 窗口-Phase2。 本窗口负责： 调研自动化 CAD/CAE pipeline 的技术方案。 --- 调研目标 明确以下问题： 1 市面上的开源 CAD 软件有什么，各自优劣如何，如何实现 2 市面上开源的划分网格的软件有什么，各自优劣如何，CAD 如何传递给网格软件 3 CAE 求解器确定使用 CalculiX 软件，告诉我其他软件的优劣，以及 CalculiX 导入 Nastran 的可行性 4 计算结果如何解析 --- 调研对象：开源 Cad 划分软件 开源网格划分软件 CalculiX 重点研究： 文件格式 接口方式 自动化能力 Python 调用方式 实现成本 --- 调研结果必须输出 1 软件能力总结 2 自动化可行方案 3 pipeline 结构建议 4 每个模块职责定义 --- 输出格式 RESEARCH NOTE 主题： 内容： 结论： 对项目影响： --- AI 工作规则 你是技术调研工程师。 目标是为 Master Control 提供技术路线建议。 不要写完整工程代码。 重点是： 技术可行性 接口方式 自动化流程

ChatGPT 说：

已思考 1m 44s

RESEARCH NOTE

主题：自动化 CAD/CAE pipeline 技术路线调研（Phase 2）

内容：

1. 开源 CAD 软件有什么，各自优劣如何，如何实现

候选 1：CadQuery
CadQuery 是基于 Python 的参数化 3D CAD 建模库，底层基于 OpenCascade/OCP，天然适合“代码生成几何”的工作方式；它支持输出 STEP、STL、AMF、3MF 等格式，定位就是脚本化、参数化建模。对你这个项目来说，它最大的优势是：直接 Python 驱动、容易和 CaseSpec 参数字典对接、适合批量生成模板件。它的短板是：偏“开发者工具”，交互式修改和复杂装配的人机操作体验不如传统 GUI CAD。
cadquery.readthedocs.io+2cadquery.readthedocs.io+2

候选 2：FreeCAD
FreeCAD 是开源参数化 3D 建模软件，基于 Open CASCADE，支持 BRep、NURBS，并可导入导出 STEP、IGES、OBJ、STL、DXF 等标准格式。它内置 Python 解释器，API 覆盖界面、几何和文档对象，因此很适合做“GUI 调试 + Python 自动化脚本”的混合路线。它的优势是生态成熟、可视化好、适合人工检查；短板是工程自动化时对象模型更重，脚本控制通常比 CadQuery 更啰嗦。freecad.org+2FreeCAD Wiki+2

候选 3：OpenSCAD
OpenSCAD 是脚本式实体建模器，适合规则几何、布尔运算和程序化零件生成。它的优点是简单、可重复；缺点是它更偏 CSG/网格导向，不是这条 CAD→CAE 工程链里最自然的 B-Rep 工程建模工具，因此不适合作为主 CAD 内核。
en.wikibooks.org+1

本项目建议
主路线建议选 **CadQuery 作为 CAD 生成核心**，FreeCAD 作为人工检查、辅助修模、可视化验证工具。原因很直接：你的上游是结构化 CaseSpec，下游是自动网格与自动求解，最重要的是“参数—几何”的稳定映射，而不是人工交互建模。CadQuery 在这点上最合适；同时它和 FreeCAD/Gmsh 都站在 OpenCascade 这条技术线上，接口更顺。cadquery.readthedocs.io+2freecad.org+2

---

2. 开源网格划分软件有什么，各自优劣如何，CAD 如何传递给网格软件

候选 1：Gmsh
Gmsh 是开源 3D 有限元网格工具，带内建 CAD 引擎和后处理模块，支持命令行、脚本语言和完整 API。它支持 OpenCASCADE 几何内核，可直接导入 STEP、IGES、BREP，并支持 Physical Groups；同时有 Python API，能做几何导入、分组、网格控制、网格导出。对自动化 pipeline 来说，这是非常关键的。Gmsh+3Gmsh+3Gmsh+3

候选 2：SALOME（GEOM + SMESH）
SALOME 的 Geometry 模块支持导入导出 STEP、IGES、BREP、STL、VTK、XAO；SMESH 模块支持导入/导出 MED、UNV、DAT、STL、GMF、CGNS 等格式，也支持 Python 脚本。它适合比较复杂的几何修复、分组和 GUI 交互式前处理。缺点是平台更重、自动化集成成本更高，不如 Gmsh 轻量。Salome Platform 文档+2Salome Platform 文档+2

候选 3：Netgen
Netgen 提供 Python 接口，可基于 OCC/STEP 导入与网格生成，适合做几何和网格实验。它有能力，但在你这条工程化链路里，文档、接口统一性和与 CalculiX 的常见组合度都不如 Gmsh。NGS-Py+1

CAD 传递给网格软件怎么做
最稳妥的路线是：**CAD 输出 STEP，Gmsh 读取 STEP，再在 Gmsh 中建立 Physical Groups 并生成网格。** Gmsh 官方文档明确支持在 OpenCASCADE 内核下导入 STEP/IGES/BREP 并继续操作；Physical Groups 则是后续把边界条件、载荷面、材料域传给求解器的关键桥梁。Gmsh+2Gmsh+2

为什么不建议 CAD 直接输出 STL 给网格软件
STL 是三角面片网格，不是工程 CAD 的拓扑实体格式。进入 CAE 前如果过早变成 STL，面、边、体、命名区域会更难稳定保留，边界条件和分组会更麻烦。因此本项目主接口不应是 STL，而应是 **STEP**。这点也和 CadQuery、FreeCAD、Gmsh 的能力匹配。cadquery.readthedocs.io+2freecad.org+2

---

3. CAE 求解器使用 CalculiX，其他软件优劣如何，以及 CalculiX 导入 Nastran 的可行性

CalculiX 本身
CalculiX 包含前后处理器 CGX 和求解器 CCX。官方说明里，CCX 使用 Abaqus input format；CGX 可以写出 mesh-related data 到 Nastran、Abaqus、Ansys、Code_Aster、OpenFOAM 等格式。CalculiX 支持线性/非线性、静力、动力、热分析。对自动化项目而言，它最大的优势是：免费、工程味足、和 .inp 工作流天然兼容。calculix.de+1

其他开源 CAE/求解器简评

Code_Aster
偏工业级多物理场/结构仿真平台，功能强，但体系更重，学习和集成成本通常高于 CalculiX。若项目目标是先打通自动化主链，而不是追求极复杂工况，Code_Aster 不是第一优先。GitHub

Elmer FEM
Elmer 擅长多物理场，支持结构、流体、传热、电磁等，适合更广泛 PDE 问题。缺点是与“机械结构件自动 CAD→网格→结构求解”这条线相比，工程使用习惯与社区范式不如 CalculiX 贴近传统有限元前后处理流。GitHub

FEniCS
FEniCS 的强项是“用 Python/C++ 快速实现 PDE 有限元”，非常适合科研算法和自定义方程，但它不是传统工业前后处理工作流里的即插即用结构求解器。若你的目标是标准工程零件自动分析，它更像研究框架，不像成品求解链。FEniCS Project

OpenFOAM
OpenFOAM 主体仍是 CFD 平台，虽然也覆盖部分固体与耦合能力，但如果你的主任务是结构有限元，不应把它作为核心求解器。OpenFOAM+1

CalculiX 导入 Nastran 的可行性
这里要分开说：

1.

CCX 的主输入格式是 Abaqus .inp。官方明确说 solver 使用 Abaqus input format。calculix.de+1

2.
3.

CGX 能写出 mesh-related data 到 Nastran，这说明 CalculiX 生态和 Nastran 网格格式有接触，但官方并没有把“直接把完整 Nastran deck 作为 CCX 主输入”列为标准主路线。calculix.de

4.
5.

因此，**“CalculiX 直接导入完整 Nastran 模型”可行性偏低，不建议作为项目主链**。如果只是已有 Nastran 网格/BDF 的节点单元数据，理论上可以通过转换链路进入 CalculiX；但若包含完整材料、载荷卡片、约束、求解控制卡，转换成本和不确定性都会上升。这是基于官方输入/输出能力做出的工程判断。calculix.de+1

6.

本项目建议
求解主接口统一为 **CalculiX** `.inp`。不要把“Nastran 兼容”设计成核心需求，只把它当“后续可能的网格/模型迁移辅助能力”。

---

4. 计算结果如何解析

CalculiX 结果链路
CGX 官方说明是：它用于生成和显示有限元模型，以及显示来自 CCX 的结果；并且可以读取 `.frd` 结果文件。也就是说，CalculiX 官方默认结果工作流是 **CCX 输出结果 → CGX 读取/显示 FRD**。[dhondt.de+2dhondt.de+2]

工程上建议的结果解析方式
不要把“结果解析”完全依赖 CGX GUI。更适合自动化项目的做法是：

-

  基础结果文件：保留 CCX 输出的 `.frd` / `.dat` / `.inp`。

-
-

  自动解析层：写 Python 解析器，抽取位移、应力、应变、反力、最大值、最小值、关键节点/单元结果。

-
-

  展示层：二选一

-
  -

    轻量方案：直接生成 CSV / JSON / Markdown 报告；

  -
  -

    可视化方案：转换到更通用格式后，用 PyVista/VTK 类工具展示。PyVista 可直接读取 VTK 支持或 meshio 支持的文件格式。[PyVista+1]

  -

结果解析最推荐的项目策略
第一版先不要追求复杂云图自动渲染，优先做三层输出：

1.

数值摘要：最大位移、最大等效应力、最危险单元、收敛信息。

2.
3.

结构化结果：JSON/CSV，供后续数据库或批量对比使用。

4.
5.

可选可视化：后续再补 PyVista 或 CGX 批处理图像导出。

6.

这样最符合“先打通 pipeline，再逐步增强”的工程节奏。

---

2. 自动化可行方案

推荐主方案

CaseSpec → CadQuery → STEP → Gmsh → CalculiX `.inp` **→ CCX → Python Result Parser → Output**

这是当前最稳、实现成本最低、自动化最顺的一条路线。
原因：

-

CaseSpec 本身就是结构化参数，最适合喂给 CadQuery。

-
-

CadQuery 输出 STEP 很自然。[cadquery.readthedocs.io+1]

-
-

Gmsh 能稳定读 STEP，并通过 Python API 自动建组、控网格、导出网格。Gmsh+1

-
-

CalculiX 以 .inp 为主输入，和工程自动生成文本文件的方式非常契合。calculix.de+1

-

不推荐方案

FreeCAD GUI 主建模 → 手工导出 → 手工网格 → 手工求解
这条路线能做，但不符合你的自动化目标。

Nastran 作为中间总线格式
不建议。因为你的求解器已经确定是 CalculiX，而它的标准入口不是 Nastran deck。统一用 .inp 更干净。

---

3. pipeline 结构建议

建议结构

Phase A：参数标准化
CaseSpec 负责把几何、材料、载荷、边界、网格参数整理成统一结构。

Phase B：几何生成
CadQuery 读取 CaseSpec，生成参数化实体模型，并输出 STEP。

Phase C：网格生成
Gmsh 读取 STEP，创建 Physical Groups，按 CaseSpec 中的网格规则生成网格，并导出 CalculiX 所需输入片段或中间网格。

Phase D：求解输入组装
CalculiX 模块负责把网格、材料、边界条件、载荷、步控制组装成完整 .inp。

Phase E：求解执行
调用 CCX 执行分析。

Phase F：结果解析输出
解析 .frd / .dat，抽取关键结果，输出 CSV / JSON / 图表 / 简报。

4. 每个模块职责定义

01 Research

负责技术路线调研、文件格式选择、接口标准定义，不写核心业务实现。

02 CaseSpec

负责统一算例数据结构。
输出内容应至少包含：

- geometry
- material
- load
- boundary
- mesh
- analysis
- naming / groups

•

它不负责几何建模，只负责“把算例说清楚”。

03 CadQuery

负责把 CaseSpec 的几何参数转成 CAD 实体。
输出：STEP。
不负责网格，不负责求解。

04 Gmsh

负责导入 STEP、建立 Physical Groups、设置网格尺寸与局部加密、导出网格与分组信息。
不负责材料本构和求解步定义。

05 CalculiX

负责生成 .inp、调用 CCX、管理求解工况、收集日志。
这里是“分析定义中心”。

06 Output

负责结果抽取、结构化输出、自动报告。
第一版优先做数值摘要，不强求复杂后处理 GUI。

结论：

1.

CAD 主工具选 CadQuery，辅助工具选 FreeCAD。

2.
3.

网格主工具选 Gmsh，不建议一开始用 SALOME 作为主线。

4.
5.

求解器主线坚定使用 CalculiX .inp，不要把 Nastran 兼容作为核心路线。

6.

7.

主接口格式统一为：CAD 用 STEP，求解输入用 `.inp`，结果用 `.frd/.dat + JSON/CSV`。

8.
9.

第一版项目重点不应是 GUI，而应是：参数结构稳定、接口清晰、自动执行成功。

10.

对项目影响：

这次调研对 Master Control 的直接建议是：

- **技术路线确定：**
  `CaseSpec → CadQuery → STEP → Gmsh → CalculiX(.inp) → Output`
-
- **接口规范确定：**
-
  - CAD 交换：`STEP`
    -
    -
  - 网格/分组：`Gmsh Physical Groups`
    -
    -
  - 求解输入：`CalculiX .inp`
    -
    -

结果输出：FRD/DAT + JSON/CSV

○

-

实施优先级确定：
先做 **02 CaseSpec** 和 **03 CadQuery** 的接口闭环，随后做 **04 Gmsh**，再进入 **05 CalculiX**。
也就是说，下一阶段最关键的不是继续泛调研，而是把 **CaseSpec → CadQuery 的最小可运行样例** 先打通。

-

如果你要，我下一步可以把这份调研继续整理成一份 **“Master Control 可直接采用的技术路线决策稿”**。

来源

你说：

2.我现在构筑的这个 pipeline 是要向商用发展的。calculiX 的结果可以很容易导入 abaqus，这点有利于 ToC 的发展。但是如果面临 ToB 时，面对复杂结构、复杂工况，我们可能需要比如 nastran 软件等的复杂结构分析能力。这时候我的 pipeline 如何构建？给我推荐。 3.将 CalculiX 计算结果进行展示，该层的选用有建议吗？国内外各个开源的可视化工具各有何优劣？我希望最终可以输出直观的应力云图、载荷位移曲线等。

你说：

1.FreeCAD 与 CadQuery 各自的优劣在哪里？我这个 pipeline 你更推荐哪种？你说辅助工具选 FreeCAD，如何辅助？ 2.我现在构筑的这个 pipeline 是要向商用发展的。calculiX 的结果可以很容易导入 abaqus，这点有利于 ToC 的发展。但是如果面临 ToB 时，面对复杂结构、复杂工况，我们可能需要比如 nastran 软件等的复杂结构分析能力。这时候我的 pipeline 如何构建？给我推荐。 3.将 CalculiX 计算结果进行展示，该层的选用有建议吗？国内外各个开源的可视化工具各有何优劣？我希望最终可以输出直观的应力云图、载荷位移曲线等。 4.我同意你“第一版项目重点不应是 GUI，而应是：参数结构稳定、接口清晰、自动执行成功。”的观点。如果按照你最新的建议，我应该率先实

现哪方面的东西？各个接口如何？注意，你不需要具体编程，你只需要告诉我输入输出是什么即可。

ChatGPT 说：

RESEARCH NOTE

主题：自动化 CAD/CAE pipeline 商用化架构调研（CAD 选择 / ToB 求解器策略 / 可视化层 / MVP 实现路线）

---

1 FreeCAD 与 CadQuery 的优劣与 pipeline 推荐

CadQuery

定位

Python 参数化 CAD 库（OpenCascade 内核）

优点

1. **完全代码驱动**
2. CAD 几何可以直接由 Python 脚本生成
   非常适合自动化 pipeline
3. **与 CaseSpec 天然匹配**
4.

```
CaseSpec(JSON/YAML)
↓
Python dict
↓
```

CadQuery
↓
CAD

非常顺畅

1.

可批量生成

2.

一次生成几千个算例没有问题

1.

接口清晰

2.

输出：

STEP
STL
AMF
3MF

1.

自动化能力极强

2.

可以直接作为

CaseSpec → CAD

的生成器

缺点

1.

没有强 GUI

2.

3.

装配支持较弱

4.
5.

几何修复能力一般

6.
7.

不适合人工建模

8.

FreeCAD

定位

完整 CAD 软件（类似开源 SolidWorks）

优点

1.

强 GUI

2.

可以手工建模

1.

Python API

2.

可以脚本控制

1.

几何修复能力

2.

导入 STEP 修复非常方便

1.

装配能力

2.

比 CadQuery 强

1.

工程生态较完整

2.

缺点

1.

自动化脚本复杂

2.
3.

对象结构重

4.
5.

batch automation 不如 CadQuery

6.

推荐方案

核心 CAD：CadQuery

辅助 CAD：FreeCAD

FreeCAD 如何辅助

主要用于三件事

1 CAD 模板设计

工程师可以用 FreeCAD 设计模板

例如

flat_plate.FCStd
stiffened_panel.FCStd
beam.FCStd

然后再转化为

CadQuery template

---

2 CAD 可视化检查

自动生成 CAD 后

人工可用 FreeCAD 打开

STEP

检查是否合理

---

3 CAD 修复

复杂模型可能需要

STEP repair
geometry healing

FreeCAD 做这个非常方便

最终 CAD 结构

推荐结构

```
CaseSpec
↓
CadQuery
↓
STEP
↓
FreeCAD (optional check)
```

---

2 商用发展 ToC + ToB pipeline 架构

这是一个非常关键的问题。

核心原则：

不要绑定求解器

而是做

solver abstraction layer

推荐架构

```
CaseSpec
↓
Geometry Generator
↓
Mesher
↓
Analysis Model
↓
Solver Adapter
↓
Solver
↓
Result Adapter
↓
Visualization
```

Solver Adapter 层

这是关键。

每个求解器做一个 Adapter

例如

CalculiXAdapter
NastranAdapter
AbaqusAdapter
CodeAsterAdapter

每个 Adapter 负责

把

analysis model

转换为

solver input file

例如

CalculiX

输出

*.inp

Nastran

输出

*.bdf

Abaqus

输出

*.inp

Analysis Model

必须存在一个

统一中间模型

例如

analysis_model.json

包含

nodes
elements
materials
loads
boundary
steps

---

pipeline 结构

推荐

CaseSpec
↓
CadQuery
↓
STEP
↓
Gmsh
↓
Mesh Model
↓
Analysis Model
↓
Solver Adapter
↓
Solver

这样带来的优势

未来支持：

CalculiX
Nastran
Abaqus
CodeAster

只需要增加

Adapter

而不需要修改 pipeline

商业架构建议

ToC

CalculiX

ToB

Nastran
Abaqus

你的 pipeline 完全可以同时支持

---

3 计算结果可视化方案

目标

输出

应力云图
位移云图
变形图
载荷-位移曲线

主流开源可视化工具

ParaView

最强

优点

1.

工业级

2.
3.

基于 VTK

4.
5.

支持

6.

vtk
vtu
exodus
ensight

1.

超大模型

2.

缺点

1 学习成本

PyVista

Python 版 VTK

优点

1 Python 友好
2 代码简洁
3 自动截图

非常适合自动化

例如

自动生成云图
自动生成 png

---

matplotlib

优点

简单

缺点

不适合 3D FE mesh

只适合

曲线
图表

Plotly

优点

网页可视化

缺点

大型 FE mesh 性能差

推荐组合

FE 云图 → PyVista
曲线 → matplotlib

输出形式

最终可以输出

stress.png
displacement.png
load_disp_curve.png
result_summary.json

商业版建议

未来可以做

web viewer

例如

vtk.js
three.js

浏览器直接看 FE 结果

---

4 第一阶段应该实现什么

你之前的判断是对的

第一阶段不要 GUI

只做

pipeline 跑通

MVP pipeline

最小版本

```
CaseSpec
↓
CadQuery
↓
STEP
↓
Gmsh
↓
mesh
↓
CalculiX
↓
result
```

第一阶段只需要做 3 件事

1 CaseSpec

输入

```
case.yaml
```

例如

```yaml
geometry:
  type: flat_plate
  length: 1
  width: 0.2
  thickness: 0.01

material:
  E: 70e9
  nu: 0.3

load:
  type: tension
  value: 1000

mesh:
  size: 0.01
```

输出

casespec.json

---

2 CAD 模块

输入

casespec.json

输出

model.step

---

3 Gmsh 模块

输入

model.step
casespec.json

输出

mesh.inp

---

4 CalculiX 模块

输入

mesh.inp
casespec.json

输出

job.frd
job.dat

---

5 Output 模块

输入

job.frd

输出

stress.png
disp.png
summary.json

MVP 接口图

```
CaseSpec
    ↓
casespec.json

CadQuery
    ↓
model.step

Gmsh
    ↓
mesh.inp

CalculiX
    ↓
job.frd

Output
    ↓
png / json
```

关键原则

整个系统必须围绕

文件接口

构建

不要一开始搞复杂 API

最关键的设计

只有两个核心接口

几何接口

STEP

分析接口

analysis model

结论

对于你的项目

最推荐架构

```
CaseSpec
↓
CadQuery
↓
STEP
↓
Gmsh
↓
Mesh
↓
Analysis Model
↓
Solver Adapter
↓
Solver
↓
Result Parser
↓
Visualization
```

如果你愿意，我可以在下一步帮你做一件**极其关键的事情：**

设计整个 pipeline 的工业级数据接口规范

包括

CaseSpec schema
AnalysisModel schema
Result schema

这会直接决定这个项目未来能不能商业化。

你说：

以下所有都给我工程级建议，首先从稳定、可维护、易扩展角度来考虑。 1.由于为了规避版权限制，因此我们cad建立的软件必须使用开源软件。但是，工程实际中catia运用明显更加广泛。如果面向ToB的项目时，工程师可以偷偷使用catia辅助设计模板，或者进行可视化检查。此时此pipiline是否兼容？或者其他软件？ 2.”不要绑定求解器“的核心原则非常好。通过Solver Adapter 层可以有效解决输出为闭源软件所需格式的问题。同时，也避免了转为CalculiX再转为Abaqus的冗余。但是我担心analysis_model.json格式是否足够兼容？比如*.inp格式和*.bdf所需的信息应该也不完全相同吧？因为，实际计算时，为了提升效率，每次面临新任务时候，我们首先从CaseSpec中找寻是否有已有模板，如果有则直接套用此模板即可。CaseSpec库输出analysis_model.json是否方便？ 3.后处理部分。我们可能的输出需求有：载荷-位移曲线；应力应变曲线；载荷、弯矩值；等。我认为你推荐的“FE云图 → PyVista；曲线 → matplotlib”很好。但是，除此以外还有什么输出类型吗？以abaqus为例，帮我尽可能多的考虑输出类型。面对某些简单的输出（如弯矩其实仅需要各方向分量大小就够了），应该如何输出这些仅数据的部分？就是用result_summary.json吗？ 4.几何接口，我不仅仅希望用户只能输入.step格式文件，最终版中，我希望用户可以使用自然语言输入、图片输入、step格式文件输入。此时我的pipeline该如何构建？当然，这一点不急着实现，我只是考虑周全。 5.我需要用户使用数据储存功能，帮我汇集用户所用过程中的数据、案例、结果等。在这个pipiline中该如何设置？当然，这一点不急着实现，我只是考虑周全。 6.我需要报错修复功能来辅助我调试代码。在这个pipiline中该如何设置？同样的，这一点不急着实现，我只是考虑周全。

ChatGPT 说：

已思考 59s

下面我按你这 6 点，给出**工程级、面向长期商用**的建议。重点只放在三件事：

稳定、可维护、易扩展。

RESEARCH NOTE

主题：自动化 CAD/CAE pipeline 的工程级架构补充建议

---

1. 面向 ToB 时，工程师若用 CATIA 或其他商业 CAD 辅助设计模板 / 做可视化检查，pipeline 是否兼容？

结论先说：兼容，而且应该主动设计成兼容。

你的主 pipeline 不应该绑定“某一个 CAD 软件”，而应该绑定**几何交换标准**。
对你这个项目，最稳的做法不是“规定只能用 CadQuery/FreeCAD”，而是：

内部建模主线用开源工具；外部协同允许任何能稳定导出标准格式的 CAD。

CadQuery 本身支持导出 STEP；FreeCAD 也支持导入导出 STEP，并且带 Python API；Gmsh 在 OpenCASCADE 内核下支持导入 STEP/IGES/BREP，并可保留 STEP/IGES 的标签信息。CATIA 侧也支持 STEP AP203/AP214 的导入导出。
也就是说，只要你把**几何主接口锁定为 STEP**，那么 CATIA、FreeCAD、CadQuery 甚至其他商用 CAD 都可以接进来。Catia Documentation+4CadQuery 文档+4FreeCAD Wiki+4

但这里有一个非常重要的工程判断：

兼容 STEP，不等于兼容“参数化特征历史”。

CadQuery 文档明确提到，外部文件格式用于交换 CAD 模型数据，但**不携带 CadQuery 自身的参数化信息**；STEP 适合交换几何，不适合做“参数模板的主存储”。所以你要把“几何交换”和“参数模板定义”彻底分开。CadQuery 文档

工程级建议

你应该把 CAD 层分成两条线：

A. 主生产线（可自动化、可批量）

- 输入：CaseSpec

-
-

生成器：CadQuery

-
-

输出：STEP

-

这条线是你真正可控、可复制、可商用的核心。

B. 外部协同线（面向 ToB 工程师）

-

输入：CATIA / SolidWorks / NX / FreeCAD 等生成的 STEP

-
-

用途：

-

    1.

    复杂模板原型设计

    2.
    3.

    复杂几何人工修补

    4.
    5.

    工程师人工审查

    6.
    7.

    客户已有 CAD 数据接入

8.

因此，推荐你正式规定：

“允许使用任何 CAD 做辅助，但进入本 pipeline 的统一几何接口必须是 STEP。”

这样做的好处

1.

不受版权限制
你的系统核心仍然依赖开源链路。

2.
3.

兼容 ToB 工程现实
客户现场几乎不可能只用你指定的 CAD。

4.
5.

维护成本低
你只维护一条 STEP 接口，不维护 N 套 CAD 原生接口。

6.
7.

风险可控
一旦 CATIA/NX/SW 版本变化，受影响的是它们导 STEP 的质量，不会直接把你整个系统拖垮。

8.

进一步建议：定义“几何接入等级”

为了稳定性，我建议你把几何输入分级：

-

L1：参数驱动输入
CaseSpec → CadQuery → STEP
这是你最推荐、最稳定的路线。

-

•

L2：标准 CAD 交换输入
外部 CAD → STEP → Gmsh
适合客户已有模型。

•
•

L3：网格级输入
外部网格 → Mesh Import
仅作为后续扩展，不建议第一版做。

•

其他软件是否兼容？

只要能稳定导出 STEP/BREP/IGES，就能进入你的几何入口。
但工程上我建议你**只把 STEP 设为正式受支持格式**，IGES/BREP 仅作为“兼容模式”。因为 STEP 在实体几何交换上通常是更主流、更稳的通道。FreeCAD 手册也明确建议：涉及实体与 NURBS 时，优先用 STEP。freecad.org

---

2. “不要绑定求解器”很好，但 `analysis_model.json` 是否足够兼容？`.inp` 和 `.bdf` 需求并不完全相同。CaseSpec 库输出它是否方便？

你的担心非常对。

如果把 `analysis_model.json` 设计得太“扁平”，它以后一定不够用。

因为 CalculiX/ Abaqus `.inp` 和 Nastran `.bdf` 的建模逻辑并不一样。
CalculiX 的主输入是 Abaqus 风格 input deck；而 Nastran/BDF 则是明显的 card/bulk-data 体系，常见条目包括 GRID、元素卡、材料卡、载荷卡、约束卡等，不是简单“节点 + 单元 + 材料 + 载荷”四个字段就能完整覆盖的。
dhondt.de+2nexus.hexagon.com+2

所以，**不要把统一中间模型理解成“一个万能 JSON 把所有求解器细节抹平”。**
正确做法是：

统一中间模型要分成三层

第一层：Problem Definition（问题定义层）

这是最上层，直接从 CaseSpec 来，表达“工程意图”。

包含：

- 几何类型
  -
  -
- 参数
  -
  -
- 材料
  -
  -
- 工况
  -
  -
- 边界条件
  -
  -
- 网格偏好
  -
  -
- 分析类型
  -
  -
- 结果需求
  -

这层不关心求解器细节。

第二层：Canonical Analysis Model（规范分析模型层）

这是你真正的“统一中间模型”。

它不是简单的 `analysis_model.json`，而应该是一个**规范分析对象模型**，至少包含：

- geometry_refs
- topology / regions / surfaces / edges / points
- mesh entities
- named sets
- section / property
- material assignment
- loads

-

boundary conditions

-
-

contact / tie / connector

-
-

analysis steps

-
-

output requests

-
-

units

-
-

metadata

-
-

provenance（来源信息）

-
-

template linkage（模板关联）

-

它要表达的是：

“这是一个完整的、求解器无关的分析问题。”

第三层：Solver Extension Layer（求解器扩展层）

这是最关键的一层。

因为规范模型不可能 100% 覆盖每个求解器的专有能力，所以每个求解器要允许挂一个扩展区：

- `calculix_extensions`
- `abaqus_extensions`
- `nastran_extensions`

例如：

- Nastran 特有的卡片控制
- Abaqus/CalculiX 的 step/output keyword 细节
- 特定接触算法参数
- 特定单元类型偏好

特定求解控制项

-

工程上最稳的规则是：

共性放 Canonical Model；个性放 Solver Extension。

不要强行全塞进统一模型里，否则最后会变成一个谁都不满意的大杂烩。

那么 CaseSpec 库直接输出 `analysis_model.json` 方不方便？

不建议 CaseSpec 直接输出最终 Analysis Model。

更稳的做法是：

CaseSpec 的职责

只负责输出：

-

问题参数

-
-

模板选择条件

-
-

目标工况

-
-

用户输入

-

Template Library 的职责

根据 CaseSpec 去匹配已有模板：

- plate_tension_template
- plate_bending_template
- stiffened_panel_buckling_template
- bolt_joint_template

Template Instantiator 的职责

把模板 + CaseSpec 参数，实例化成 **Canonical Analysis Model**

为什么这样更稳？

因为你前面说得很对：
实际工程里，为了效率，遇到新任务时，往往先看**有没有可复用模板**。

这意味着系统真正的核心不是“CaseSpec 直接生成分析文件”，而是：

CaseSpec → 模板匹配 → 模板实例化 → 规范分析模型 → Solver Adapter

这条链更适合商用发展

因为：

1. **复用性强**

2.
3.

模板版本可控

4.
5.

便于审查

6.
7.

便于支持多个求解器

8.
9.

便于形成行业知识库

10.

我对这一层的正式建议

不要只定义一个 `analysis_model.json`

而要定义三个对象：

1. `case_spec.json`

问题定义

2. `template_manifest.json`

模板能力声明
说明该模板支持什么几何、什么工况、什么求解器

3. `analysis_model.json`

模板实例化后的规范分析模型

必要时再加：

4. **solver_job.json**

某次求解任务的求解器具体配置

---

3. 后处理部分：除了云图和曲线，还应考虑哪些输出？仅数据的部分如何输出？

你提到的思路是对的：

- FE 云图：PyVista
-
-
- 曲线：matplotlib
-

这个组合非常适合自动化。PyVista 基于 VTK 体系，适合位移/应力云图、变形显示等；matplotlib 适合载荷-位移、应力-应变、时间历程等二维图。PyVista 也支持读取多种 VTK/Exodus 数据并进行变形显示。PyVista

但工程上，后处理不能只分“图”和“曲线”。
真正稳定的后处理层，应该分成**四类输出**：

A. 标量摘要输出（Summary Outputs）

这是最常用、最适合自动化汇总的结果。

例如：

- 最大位移
-
-
- 最大主应力 / Mises

-
-

最大应变

-
-

最大反力

-
-

最大弯矩 / 扭矩

-
-

质量

-
-

刚度

-
-

安全系数

-
-

临界屈曲因子

-
-

首阶频率

-
-

收敛状态

-

- 计算时间
- 这类结果不需要画图，最适合写入：
- `result_summary.json`
-
- `result_summary.csv`
-

所以你问“弯矩这种仅需要各方向分量大小的结果怎么办”——答案是：
对，这类结果就应该进入 summary schema，而不是强行做成图。

B. 场输出（Field Outputs）

这是“云图”对应的结果类型。

参考 Abaqus/CalculiX 习惯，典型包括：

- 位移 U
-
- 应力 S
-
- 应变 E / LE / PE
-

反力 RF

-
-

接触压力

-
-

温度

-
-

壳单元上下表面结果

-
-

单元/节点场变量

-
-

损伤变量

-
-

层合板逐层结果（后续如果你做复材会非常关键）

-

Abaqus 也明确区分了 field output 和 history output，并支持从 ODB 场输出和历史输出中读取 XY 数据。VT 软件文档+1

C. 历程输出（History Outputs）

这类结果是“曲线”的来源。

例如：

-

载荷-位移

-
-

应力-应变

-
-

力-时间

-
-

位移-时间

-
-

反力-时间

-
-

能量演化

-
-

接触状态演化

-
-

非线性迭代收敛曲线

-

这类结果建议输出为：

-

`history_outputs.csv`

-
-

history_outputs.parquet

-
-

plots/*.png

-

D. 报告输出（Report Outputs）

给人看的，不是给程序看的。

例如：

-

PDF 报告

-
-

Word 报告

-
-

HTML 报告

-
-

项目页 summary

-

包含：

-

任务信息

-
-

模型截图

-
-

工况摘要

-
-

关键结果表

-
-

云图图片

-
-

曲线图片

-
-

结论建议

-

以 Abaqus 思路尽可能多考虑输出类型

你提到“以 Abaqus 为例，尽可能多考虑”。从工程习惯出发，你后处理层建议至少预留这些输出类别：

结构静力

-

位移

-
-

转角

-
-

反力

-
-

应力

-
-

应变

-
-

主应力

-
-

等效应力

-
-

接触力

-
-

接触压力

-
-

接触开闭状态

-

壳/板

-

膜力

-
-

弯矩

-
-

剪力

-
-

上下表面应力

-
-

厚度方向积分量

-
-

层合板各铺层结果

-

非线性

-

塑性应变

-
-

累积塑性应变

-
-

接触穿透

-

-

迭代收敛历史

-
-

弧长/增量信息

-

屈曲

-

特征值

-
-

模态形状

-
-

临界因子

-

模态/频响

-

固有频率

-
-

振型

-
-

响应幅值

-

-

相位

-

疲劳 / 损伤（后续）

-

损伤变量

-
-

循环计数指标

-
-

寿命估计

-

热 / 热结构（后续）

-

温度场

-
-

热流

-
-

热应变

-
-

热应力

-

后处理层正式建议：不要只有一个 `result_summary.json`

建议至少拆成：

1. **`result_summary.json`**

关键标量摘要

2. **`field_manifest.json`**

有哪些场结果、在哪些步、哪些帧、哪些区域

3. **`history_data.csv` / `history_data.parquet`**

曲线原始数据

4. **`artifacts/`**

图片与报告文件

- stress_vm.png
- displacement_mag.png
- load_disp.png
- report.pdf

5. **`diagnostics.json`**

收敛、告警、异常、求解质量

---

4. 最终版希望支持自然语言输入、图片输入、STEP 输入。pipeline 如何构建？

这一步现在不急，但你现在就要把架构留好口。

千万不要把“输入方式”直接绑到 CAD 内核。

正确做法是把系统分成：

输入理解层（Input Understanding Layer）

负责把不同类型输入统一转成**结构化需求**

支持未来三类入口：

A. 自然语言输入

例如：
“做一个长 500、宽 100、厚 4 的铝合金平板，两端固支，中间施加向下集中载荷。”

输出：
`intent_spec.json`

B. 图片输入

例如：

- 草图
- 截图
- 简单零件照片

-
-

标注图

-

输出仍然不是直接 CAD，而是：
`intent_spec.json` 或 `geometry_hint.json`

C. 直接 STEP 输入

这时跳过几何生成，只做：

-

几何识别

-
-

分区命名

-
-

后续网格与分析建模

-

推荐的总结构

层 1：Input Adapter

把不同输入转成统一中间描述：

-

`nl_adapter`

-
-

`image_adapter`

-
-

`cad_file_adapter`

-

层 2：Intent / Geometry Spec

输出统一结构化描述：

-

几何目标

-
-

尺寸

-
-

特征

-
-

工况

-
-

材料

-
-

约束

-
-

置信度

-

层 3：Geometry Realization

决定怎么落地：

-

若是参数化可生成件 → CadQuery

-
-

若是用户给 STEP → 直接走 Gmsh

-
-

若是图片/文本但不够完整 → 进入人工补全流程

-

工程上最稳的原则

不要承诺“自然语言/图片 → 100% 自动精确 CAD”

这在工程上不稳。

你应该设计成：

自然语言 / 图片 → 结构化草案 → 人工确认 / 自动补全 → 几何生成

这样才可维护。

---

5. 需要用户数据存储功能：汇集使用过程中的数据、案例、结果。pipeline 该如何设置？

这个需求对 ToB/ToC 都非常关键，而且你现在就该留架构。

推荐原则：

把“求解 pipeline”和“数据平台”分开。

不要让主求解流程直接依赖数据库逻辑，否则后期会越来越乱。

建议引入“Project Data Layer”

分为 5 类对象：

1. Case Record

一次案例定义

- 用户输入
  - CaseSpec
  - 模板版本
  - 几何版本

2. Job Record

一次求解任务

- 求解器
  -

提交时间

-
-

状态

-
-

输入文件

-
-

输出文件

-
-

日志

-

3. Result Record

一次求解结果

-

关键指标

-
-

输出曲线

-
-

云图资源

-
-

结果摘要

-

4. Artifact Record

附属文件

-

STEP

-
-

网格

-
-

报告

-
-

PNG

-
-

CSV

-
-

PDF

-

5. Knowledge Record

可复用知识

-

模板

-

-

常见失败模式

-
-

修复经验

-
-

推荐网格策略

-

存储建议

热数据

关系型数据库保存元数据：

-

PostgreSQL

-

冷数据 / 大文件

对象存储：

-

MinIO / S3 兼容存储

-

搜索

-

Elasticsearch / OpenSearch（后续）
用于检索案例、日志、报错模式

-

你最终应支持的能力

1.

按项目查看历史案例

2.
3.

按几何类型查相似案例

4.
5.

按工况查历史结果

6.
7.

按失败日志查修复经验

8.
9.

对比不同求解器结果

10.
11.

对比模板版本结果差异

12.

换句话说：

你不只是做一个“自动求解器”，
而是在做一个**工程分析资产平台**。

---

6. 需要报错修复功能来辅助调试代码。pipeline 中该如何设置？

这个也很关键，而且越早设计越省事。

正式建议：

不要把“报错修复”理解成简单日志打印。
你需要的是一个**可诊断的** pipeline。

应该加入 4 层诊断能力

A. 输入校验层

在每一模块入口做 schema 校验：

- CaseSpec 校验
- 几何参数合法性校验
- 材料参数校验
- 工况完整性校验
- 单位校验

一旦失败，尽量不要等到求解器再报错。

B. 接口校验层

每个模块的输入输出都做验证：

- STEP 是否存在
- STEP 是否可读
- Gmsh Physical Groups 是否齐全
- 网格质量是否达标
- 分组是否丢失
- Analysis Model 是否完整
- Solver input 是否通过预检查

CalculiX 官方就有用于输入检查/几何检查的流程，CCX 也会输出 `.dat` 与 `.frd` 等结果文件；这些日志和检查结果都应接入你的诊断系统。[Massachusetts Institute of Technology+1]

C. 运行诊断层

每次 job 都记录：

- stdout / stderr
- 运行时长
- 内存占用
- 网格规模
- 单元类型
- 失败模块
- 错误码
- 重试记录

D. 修复建议层

这是你说的“辅助调试代码”的真正落点。

针对常见错误做规则库：

例如：

-

STEP 导入失败
→ 检查是否为装配体、是否丢失实体、是否需几何修复

-
-

Physical Groups 缺失
→ 检查命名面是否存在、模板映射是否正确

-
-

网格质量差
→ 建议局部加密 / 曲率网格 / 尺寸回退

-
-

求解不收敛
→ 建议减小步长 / 修改接触 / 检查约束刚体模态

-
-

结果异常大
→ 检查单位、载荷方向、约束缺失

-

推荐你新增两个系统对象

1. `diagnostic_report.json`

记录：

-

错误位置

-
-

错误类型

-
-

原始日志

-
-

推断原因

-
-

修复建议

-
-

关联案例

-

2. **`health_checks.json`**

记录：

-

几何健康度

-
-

网格健康度

-
-

模型完整性

-
-

求解稳定性

-
-

结果可信度

-

对调试代码本身的建议

你以后每个模块都应输出三类结果：

主输出

例如 STEP、mesh、inp、png

日志输出

机器可读 + 人可读

诊断输出

结构化 JSON

也就是说，每个模块都不是只返回一个文件，而是返回一个“结果包”：

-

artifact

-
-

log

-

- diagnostics
- metadata

这会极大提高可维护性。

最终工程级建议汇总

你这条 pipeline，建议升级成下面这套结构：

1. 输入层

- 自然语言输入
- 图片输入
- STEP 输入
- 参数输入

2. 问题定义层

case_spec.json

-

3. 模板匹配层

-

模板检索

-
-

模板版本控制

-
-

模板实例化

-

4. 规范分析模型层

-

analysis_model.json

-
-

solver-specific extensions

-

5. 几何层

-

CadQuery 主生成

-
-

STEP 统一交换

-

-

FreeCAD / CATIA / 其他 CAD 作为辅助协同工具

-

6. 网格层

-

Gmsh 主线

-
-

命名区域 / Physical Groups

-
-

网格质量检查

-

7. 求解器适配层

-

CalculiXAdapter

-
-

AbaqusAdapter

-
-

NastranAdapter

-
-

后续扩展

-

8. 后处理层

-

Summary

-
-

Field

-
-

History

-
-

Report

-
-

Diagnostics

-

9. 数据资产层

-

案例

-
-

Job

-
-

结果

-
-

报告

-
-

模板

-
-

失败经验库

-

10. **诊断层**

-

校验

-
-

日志

-
-

自动修复建议

-
-

健康检查

-

结论

你现在最该坚持的三条总原则：

第一，几何接口标准化。
所有 CAD，无论开源还是闭源，只要进入系统，就统一成 STEP。Catia Documentation+3CadQuery 文档+3Gmsh+3

第二，分析模型分层。
不要幻想一个简陋 `analysis_model.json` 同时完美覆盖 `.inp` 与 `.bdf`；要做“规范模型 + 求解器扩展”。dhondt.de+2nexus.hexagon.com+2

第三，后处理与数据平台前置留口。
云图和曲线只是表面，真正可商用的是“结果资产化、可追溯、可复用、可诊断”。

下一步最值得做的，不是编程，而是先把这 6 层里的**数据接口规范**定下来。
我建议下一步直接进入：

“工程级接口规范 V1”
我可以按你的项目结构，给你写出一份：

- `case_spec.json` 字段规范
- `template_manifest.json` 字段规范
- `analysis_model.json` 分层规范
- `result_summary.json` / `diagnostic_report.json` 规范

这样你后面无论接 CalculiX、Abaqus 还是 Nastran，都会稳很多。

来源

你说：

1.我同意你的观点“你应该把 CAD 层分成两条线：A. 主生产线（可自动化、可批量）B. 外部协同线（面向 ToB 工程师）”。予以保留。 2.我了解 abaqus 风格的 input deck，但不怎么了解 Nastran 的 card/bulk-data 体系，帮我简单介绍它。你这个将“统一中间模型要分成三层”的观点，是否会导致建模信息过于冗余？第一层很好，因为最终问题描述也需要输出报告，这层予以保留；我担心的是第二次加上第三层之后，仍然要转化为专业 CAE 软件可读取的格式，会不会太复杂？有没有更简单的模式？比如能不能选择一个同时兼容 abaqus 和 nastran 的求解器？换言之，如果我要同时面向 ToB 和 ToC，那”不要绑定求解器“的核心原则就是最优的吗？ 3.“不要只定义一个 analysis_model.json 而要定义三个对象：”的建议，是不是和之前有点重复了？我想的是，analysis_model 来源应该只有两个：1）如果 CaseSpec 中已有模板，则套用；2）如果没有，则根据用户输入来重新生成。 4.后处理部分的“1.result_summary.json2. field_manifest.json3. history_data.csv / history_data.parquet4. artifacts5. diagnostics.json”思路很好，予以保留。你“后处理层建议至少预留这些输出类别”给我的建议也予以保留。 5.三层输入的总结构我是不是可以这样理解：“层 1：Input Adapter”负责识别转化，将不同的需求转为程序可识别的要求；“层 2：Intent / Geometry Spec”负责输出任务需求，将可识别的要求输出为任务结构描述；“层 3：Geometry Realization”负责实现 CAD 自动建模。如果是这样，我同意你的观点，予以保留。另外，我同意你的“自然语言 / 图片 → 结构化草案 → 人工确认 / 自动补全 → 几何生成”的流程，以及“不要承诺“自然语言/图片 → 100% 自动精确 CAD””的观点。予以保留。 6.我不同意你“在做一个工程分析资产平台”的观点，你认为我最终应当支持的能力“按项目查看历史案例、按几何类型查相似案例、按工况查历史结果、按失败日志查修复经验、对比不同求解器结果、对比模板版本结果差异”我认为仅仅部分有必要。别人具体工程分析过程不会让我们知道，分析别人的工程过程没有意义。我现在和你探讨的就是一个“AutoCAD/CAE”，需要的只是快捷输入需求输出 CAD/CAE 结果。用户数据存储最终目的是写入我们的 CaseSpec 库中。只是，我觉得应当加一道审查过程。此时的 pipeline 应当如何构筑？ 7.我同意你“4 层诊断能力”的观点。予以保留。 我们刚刚讨论了这么多，遍历你我的所有对话，然后，你把所有 pipeline 每个模块的具体实现方式写成 word 给我，要求特别详细，不需要如何实现，但是要告诉我哪个模块负责完成什么任务。要求命名规范、稳定、可维护、容易扩展。我先看看现在的 pipeline 是否符合我的预期。

ChatGPT 说：

已思考 15m 4s

已整理成 Word 文档：

下载 Word 文档

文档内容是基于我们这几轮关于 pipeline 的讨论汇总而成，重点按“稳定、可维护、易扩展”来写，明确了：

-

各模块负责什么

-
-

每层输入输出是什么

-
-

哪些观点已确认保留

-
-

审查、存储、诊断应如何嵌入主流程

-

你先看这版结构是否符合预期。

你说：

我给你的意见你还没回复我，先回答我。之后我们再总结。 刚刚的意见如下：1.我同意你的观点“你应该把 CAD 层分成两条线：A. 主生产线（可自动化、可批量）B. 外部协同线（面向 ToB 工程师）”。予以保留。 2.我了解 abaqus 风格的 input deck，但不怎么了解 Nastran 的 card/bulk-data 体系，帮我简单介绍它。你这个将“统一中间模型要分成三层”的观点，是否会导致建模信息过于冗余？第一层很好，因为最终问题描述也需要输出报告，这层予以保留；我担心的是第二次加上第三层之后，仍然要转化为专业 CAE 软件可读取的格式，会不会太复杂？有没有更简单的模式？比如能不能选择一个同时兼容 abaqus 和 nastran 的求解器？换言之，如果我要同时面向 ToB 和 ToC，那”不要绑定求解器“的核心原则就是最优的吗？ 3.“不要只定义一个 analysis_model.json 而要定义三个对象：”的建议，是不是和之前有点重复了？我想的是，analysis_model 来源应该只有两个：1）如果 CaseSpec 中已有

模板，则套用；2）如果没有，则根据用户输入来重新生成。 4.后处理部分的“1.result_summary.json2. field_manifest.json3. history_data.csv / history_data.parquet4. artifacts5. diagnostics.json”思路很好，予以保留。你“后处理层建议至少预留这些输出类别”给我的建议也予以保留。 5.三层输入的总结构我是不是可以这样理解：“层 1：Input Adapter”负责识别转化，将不同的需求转为程序可识别的要求；“层 2：Intent / Geometry Spec”负责输出任务需求，将可识别的要求输出为任务结构描述；“层 3：Geometry Realization”负责实现 CAD 自动建模。如果是这样，我同意你的观点，予以保留。另外，我同意你的“自然语言 / 图片 → 结构化草案 → 人工确认 / 自动补全 → 几何生成”的流程，以及“不要承诺“自然语言/图片 → 100% 自动精确 CAD””的观点。予以保留。 6.我不同意你“在做一个工程分析资产平台”的观点，你认为我最终应当支持的能力“按项目查看历史案例、按几何类型查相似案例、按工况查历史结果、按失败日志查修复经验、对比不同求解器结果、对比模板版本结果差异”我认为仅仅部分有必要。别人具体工程分析过程不会让我们知道，分析别人的工程过程没有意义。我现在和你探讨的就是一个“AutoCAD/CAE”，需要的只是快捷输入需求输出 CAD/CAE 结果。用户数据存储最终目的是写入我们的 CaseSpec 库中。只是，我觉得应当加一道审查过程。此时的 pipeline 应当如何构筑？ 7.我同意你“4 层诊断能力”的观点。予以保留。

ChatGPT 说：

已思考 13s

好，先逐条回应你的意见，并把该保留、该修正、该简化的地方说清楚。

---

1. CAD 层分成“两条线”，保留

这个判断我同意，而且我建议正式定为项目原则。

保留后的表述建议

CAD 层双轨制：

- A. **主生产线**：面向自动化、批量、可控生成
  `CaseSpec → CadQuery → STEP`
-
-

B. **外部协同线**：面向 ToB 工程师、客户已有模型、复杂模板辅助
CATIA / FreeCAD / 其他 CAD → STEP → 后续流程

-

这样定下来以后，你的系统核心只维护**一个统一几何接口 STEP**，而不是维护多套 CAD 原生接口。CadQuery 和 FreeCAD 都支持 STEP 交换，Gmsh 也支持在 OpenCASCADE 路线上导入 STEP/IGES/BREP。CalculiX 则主要走 Abaqus 风格输入。calculix.de+1

---

2. 简单介绍 Nastran 的 card / bulk-data 体系；以及“三层统一中间模型”是否过重

你这个质疑非常关键。这里我先把 Nastran 讲清楚，再回答“要不要三层”。

2.1 Nastran 的输入文件，直观上是什么

和 Abaqus/CalculiX 常见的 *NODE, *ELEMENT, *MATERIAL 这种 keyword 风格不同，Nastran 更像是：

“很多固定格式的卡片（cards / entries）拼成的输入 deck”

通常分成几块：

A. Executive Control

决定这次跑什么求解类型、调用什么解法流程。
MSC/Hexagon 文档把它单独列为 Executive Control section。
nexus.hexagon.com+1

B. Case Control

决定这个工况用哪个载荷集、哪个约束集、输出什么结果。
例如选择 LOAD、SPC、DISPLACEMENT、FORCE 等。官方文档明确说明：很多 Bulk Data 条目要被 Case Control 选中才会在当前分析中生效。
documentation-be.hexagon.com+2Autodesk 帮助+2

C. Bulk Data

这里才是真正的模型主体。
包括：

-

GRID：节点

-
-

CBEAM / CQUAD4 / CTETRA 等：单元

-
-

MAT1 等：材料

-
-

PSHELL / PBAR 等：截面/属性

-
-

SPC / MPC：约束

-
-

FORCE / MOMENT / LOAD / TABLED1：载荷与时程

-
-

还有很多求解控制、接触、模态、频响相关卡片。
Autodesk/MSC 的参考手册都把这些列在 Bulk Data section 里。
Autodesk 帮助+3Autodesk 帮助+3Autodesk 帮助+3

-

你可以把它粗略理解成

Abaqus/CalculiX 更像：

关键字块式输入

Nastran 更像：

卡片表格式输入，Case Control 去“挑选”哪些载荷集、约束集、输出集参与本次子工况

2.2 三层模型会不会太冗余？

如果严格做成完整三层，而且每层都落一个完整大文件，确实可能偏重。

所以这里我要修正我之前的建议：

对你现在这个项目，更好的工程做法不是“硬做三层全量落盘”

而是做成：

简化版两层半模型

第 1 层：Problem Definition

也就是你认可要保留的那层。
这个必须保留，因为它既是用户任务描述，也是报告输入来源。

建议文件：

- `case_spec.json`
-

第 2 层：Canonical Analysis Model

这是内部运行用的规范分析模型。
但它**不一定每次都需要作为一个超大 JSON 长期保存**，它可以是：

- 内部对象
  -
  -
- 运行时结构
  -
  -
- 必要时再序列化成 `analysis_model.json`

-

第 2.5 层：Solver Adapter Extension

我不建议把它再单独做成一个庞大的“第三主文件层”。
更实际的方式是：

每个求解器 adapter 在生成输入文件时，读取：

-

通用分析模型

-
-

少量求解器扩展参数

-

例如：

-

```
solver_profile = calculix_basic
```

-
-

```
solver_profile = nastran_linear_static
```

-
-

```
solver_profile = abaqus_contact_nonlinear
```

-

这样第三层就不是一个庞大独立体系，而是：

“在适配时附加少量求解器特有配置”

所以，修正版结论是：

你担心是对的。
完整三层理论上最通用，但对你当前阶段太重。
更适合你的是：

Problem Definition + Canonical Model + 轻量 Solver Extension

这比“纯两层”更安全，也比“重三层”更简洁。

2.3 有没有一个同时兼容 Abaqus 和 Nastran 的求解器，能省掉适配复杂度？

没有一个现实上可直接替代二者、并同时完整兼容二者输入范式的“统一求解器”，可以让你彻底省掉适配层。

原因不是“能不能读格式”这么简单，而是：

- 输入语法不同
- 单元体系不同
- 截面/属性组织方式不同
- 子工况组织方式不同
- 输出类型和求解控制方式不同

CalculiX 官方明确说明它主要使用 **Abaqus input format**。这使它和 Abaqus 风格更近，但并不意味着它天然就是 “Abaqus + Nastran 双兼容统一内核”。calculix.de+1

所以，“选一个兼容二者的求解器”不是好路线

真正更稳的路线仍然是：

不要绑定求解器，但也不要过度抽象。

更准确地说：

你要坚持的不是教条式“绝对不绑定求解器”，而是：

核心业务逻辑不绑定求解器；求解输出通过 Adapter 绑定具体求解器。

这才是对 ToB + ToC 同时成立的最优解。

为什么我仍然建议保留这个核心原则

因为你的目标已经不是“只跑 CalculiX 的个人工具”，而是：

- ToC：偏轻量、快速、便宜
- ToB：可能要求 Nastran / Abaqus / 更复杂工况

如果你现在就把分析模型写死成某个求解器思维，后面几乎一定要重构。
所以这个原则仍然应该保留，只是表达上要改成：

正式建议表述

“主流程不绑定求解器；求解器差异通过 Adapter 层吸收。”

我建议这句话予以保留。

---

3. “不要只定义一个 analysis_model.json，而要定义三个对象”是不是重复了？

你这个判断也对，我这里做修正。

你现在真正关心的是：

analysis_model 的来源只有两类

1.

CaseSpec 命中已有模板 → 套模板

2.
3.

CaseSpec 没命中模板 → 根据输入重新生成

4.

这个理解是正确的，而且更贴近工程实施。

所以我建议你把之前那种“很多对象并列”的说法，收束成更清晰的流程：

更简洁的对象关系

1. **case_spec.json**

用户任务定义 / 问题定义

2. **template_manifest.json**

模板库检索与匹配信息
它不是每次都必须独立暴露给用户，但系统内部要有

3. **analysis_model.json**

模板实例化后的分析模型
它的来源只有两种：

-

模板实例化

-
-

无模板时新建

-

所以你的说法可以正式定为：

analysis_model 不是用户直接输入的对象，而是系统生成对象。

这个观点我支持，并建议保留。

换句话说

以前我说“三个对象”，偏设计视角；
你现在说“analysis_model 的来源只有两种”，偏流程视角。

两者并不冲突，但你的说法更适合落地。

---

4. 后处理部分：你的保留意见，我同意

你保留这套结构是对的，我也建议正式保留：

- `result_summary.json`
- `field_manifest.json`
- `history_data.csv / history_data.parquet`
- `artifacts/`
- `diagnostics.json`

以及输出类别预留：

-

标量摘要

-
-

场输出

-
-

历程输出

-
-

报告输出

-

这套后处理结构对稳定性和扩展性都比较好，应该保留。

---

5. 三层输入结构：你的理解是正确的，予以确认

你现在的理解是对的，我帮你把定义再压实一点：

层 1：Input Adapter

负责“识别与转译”

-

自然语言

-
-

图片

-
-

STEP

-

- 表单/参数输入
- 统一转成程序可处理的结构化输入草案。

层 2：Intent / Geometry Spec

负责“任务定义”
输出：

- 几何目标
  -
  -
- 特征约束
  -
  -
- 材料/工况需求
  -
  -
- 输入完整度
  -
  -
- 待确认项
  - 也就是你说的“任务结构描述”。

层 3：Geometry Realization

负责“几何落地”

-

可参数化的 → CadQuery 自动建模

-
-

已给 STEP 的 → 直接进入后续流程

-
-

信息不足的 → 人工确认 / 自动补全后再建模

-

你的保留意见，我完全同意

尤其这两句应正式保留：

-

“自然语言 / 图片 → 结构化草案 → 人工确认 / 自动补全 → 几何生成”

-
-

“不要承诺 自然语言/图片 → 100% 自动精确 CAD”

-

这两句都很工程化，建议直接纳入总原则。

---

6. 你不同意“工程分析资产平台”这个说法，希望数据最终写入 CaseSpec 库，并增加一道审查过程。此时 pipeline 如何构筑？

这个我接受，并且我认为你的修正更符合你当前项目目标。

你现在做的不是一个通用仿真管理平台，
而是一个：

Auto CAD/CAE 生成系统

目标是：

- 快捷输入需求
- 自动输出 CAD / CAE 结果
- 把有价值的案例沉淀回 CaseSpec / Template Library

所以我把之前“数据平台”的说法收缩，改成更符合你目标的版本：

你真正需要的不是“资产平台”

而是：

CaseSpec / Template 增量沉淀机制

也就是：
用户每次使用系统，不是为了给你建立一个大而全的项目档案系统；
而是为了在流程结束后，判断这次案例是否值得进入你的模板/算例库。

因此，建议加入“审查层（Review Gate）”

新的主流程建议变成：

输入 → CaseSpec 生成 → 模板匹配 → analysis_model 生成 → CAD → 网格 → 求解 → 后处理 → 审查 → 选择性入库

这里的“审查”不是可有可无

而应该是正式模块。

审查层负责什么？

A. 结果审查

检查：

- 求解是否成功
- 结果是否合理
- 单位是否正确
- 关键值是否异常
- 云图/曲线是否可解释

B. 模板复用价值审查

检查这次任务是否适合沉淀为：

- 新 CaseSpec 模板
- 新几何模板

-

新工况模板

-
-

新求解配置模板

-

C. 入库审查

决定：

-

不入库

-
-

只保存运行记录

-
-

保存为候选模板

-
-

保存为正式模板

-

这样，数据存储就不再是“存一切”

而是分成两类

1. 运行级临时记录

仅服务本次任务：

- job logs
- 临时结果
- 诊断文件

2. 审查通过后的知识沉淀

只把有价值的内容写回：

- CaseSpec Library
- Template Library
- Solver Profiles
- Postprocess Profiles

这样更符合你的目标

你不是想分析别人全过程，而是想：
让系统越来越会做题。

这个方向我同意，而且建议正式保留。

6.1 审查层放在哪？

建议放在后处理之后、入库之前：

```
... → Output → Review Gate → Library Update
```

为什么放这里最好

因为这时你已经有：

- 求解状态
- 关键结果
- 诊断信息
- 云图/曲线
- 输入定义
- 模板命中情况

足够做审查。

6.2 审查层输出什么？

建议输出两个对象：

`review_report.json`

记录：

- 本次任务是否通过审查
- 通过/不通过原因
- 关键结果是否可信
- 是否建议入库
- 建议入库类型

`library_update_request.json`

如果通过审查，生成：

- 是否新增模板
- 是否更新已有模板

-
-

是否新增 solver_profile

-
-

是否新增 postprocess_profile

-

---

7. 四层诊断能力，你同意，保留

这个我同意，应该正式保留。
我建议它成为贯穿全流程的横向能力，而不是单独一个孤立模块。

四层保留如下

-

输入校验层

-
-

接口校验层

-
-

运行诊断层

-
-

修复建议层

-

这套东西对你后面调试 CaseSpec、CadQuery、Gmsh、CalculiX、Adapter 都会非常有用。

基于你这轮意见，我给出一版“修正后的更简洁 pipeline”

主流程

```
Input Adapter → Intent / Geometry Spec → CaseSpec → Template Match → Analysis Model → CAD → Mesh → Solver Adapter → Solver → Postprocess → Review Gate → Library Update
```

各层作用简述

1. Input Adapter

识别输入来源并转成结构化草案

2. Intent / Geometry Spec

定义任务需求与待确认项

3. CaseSpec

形成标准化问题描述

4. Template Match

判断是否命中已有模板

5. Analysis Model

若命中模板则实例化；未命中则新建

6. CAD

CadQuery 主生产线；外部 STEP 协同线

7. Mesh

Gmsh 生成网格与命名区域

8. Solver Adapter

把分析模型转成具体求解器输入

- CalculiX `.inp`
- Nastran `.bdf`
- Abaqus `.inp`

9. Solver

执行计算

10. Postprocess

输出：

- summary
- field
- history
- artifacts

- diagnostics
-

11. Review Gate

人工/规则审查本次案例质量与入库价值

12. Library Update

只把通过审查的内容写入 CaseSpec / Template 库

最终回应总结

你这轮意见里，我建议正式保留的内容是：

1. **CAD 双轨制**：主生产线 + 外部协同线
2. **Problem Definition 层保留**
3. **analysis_model 来源只有两种**：命中模板 / 重新生成
4. **后处理五类输出结构保留**
5. **三层输入理解结构保留**
6.

11.

自然语言/图片先转结构化草案，再确认再建模

12.
13.

四层诊断能力保留

14.
15.

增加 Review Gate，数据只选择性写回 CaseSpec / Template 库

16.

我建议修正而不是原样保留的内容是：

1.

“三层统一中间模型”太重
改为：Problem Definition + Canonical Model + 轻量 Solver Extension

2.
3.

“工程分析资产平台”表述不适合你当前目标
改为：CaseSpec / Template 增量沉淀机制 + 审查入库

4.
5.

“不要绑定求解器”要更精确
改为：主流程不绑定求解器；差异由 Solver Adapter 吸收

6.

如果你认可这版修正，我下一条就不再扩展讨论，直接把**当前最终版 pipeline 共识**整理成一份清晰的结构清单。

来源

你说：

我认可这版修正。现在，将我们上述所有讨论所得的 pipeline 形成完整、详细的 word 文档给我，包含各模块命名及功能，你我共同认可的设计准则，以及其他你觉得重要的说明。 要求命名规范、稳定、易于扩展，尽可能详细，包括模块内部细则。但是，记住，不需要真正实现，我只要知道输入输出即可。

ChatGPT 说：

我已经重新生成了文档。

你可以在这里下载：

下载 Word 文档

这份文档包含：

1 总体目标

解释整个 Auto CAD/CAE 自动化系统的目标与定位

2 我们共同确认的核心设计准则

例如：

- 主流程不绑定求解器
- CAD 双轨制（CadQuery 主生产线 + 外部 CAD 协同线）
- STEP 统一几何接口
- Analysis Model 由系统生成

-
-

模板优先策略

-
-

Review Gate 审查机制

-
-

四层诊断系统

-
-

文件接口解耦原则

-

3 完整 Pipeline 架构

完整流程：

```
Input Adapter
↓
Intent / Geometry Spec
↓
CaseSpec
↓
Template Match
↓
Analysis Model
↓
CAD
↓
Mesh
↓
Solver Adapter
↓
Solver
↓
Postprocess
```

↓
Review Gate
↓
Library Update

4 每个模块详细职责

包括：

- 模块作用
- 输入文件
- 输出文件
- 模块内部逻辑说明

例如：

CAD 模块

输入

case_spec.json

输出

model.step

说明

-

主生产线：CadQuery 自动建模

-
-

外部协同线：CATIA / FreeCAD → STEP

-

5　后处理结构（你认可的版本）

```
result_summary.json
field_manifest.json
history_data.csv
artifacts/
diagnostics.json
```

6　Review Gate 审查机制

```
review_report.json
library_update_request.json
```

决定是否写入：

```
CaseSpec Library
Template Library
Solver Profiles
```

7　四层诊断系统

1　输入校验
2　接口校验
3　运行诊断
4　修复建议

如果你愿意，我下一步可以帮你做一件**非常关键的事**（很多自动化 CAE 项目最后失败都在这里）：

我可以帮你设计：

完整工业级数据接口规范

包括：

case_spec schema
analysis_model schema
mesh schema
solver_adapter schema
result schema
diagnostic schema
review schema

这一步其实会直接决定：

你的 Auto CAD/CAE 能不能真正长期维护和扩展。

你说：

你发给我的word我打不开，想办法发给我，要求可以编辑，而且内容是中文

ChatGPT 说：

我已经重新生成了一个 **可直接用 Word 打开并编辑的文档（RTF 格式）**，里面是 **完整中文内容**。

请在这里下载：

下载文档

RTF 是 Word 原生支持的格式，特点：

- 可以 **直接用 Word 打开**
- **完全可编辑**
- 不会乱码

-

不依赖特殊插件

-
-

Windows / Mac Word 都支持

-

文档内容包含：

1 项目目标

Auto CAD/CAE 自动化系统整体目标

2 我们共同确认的核心设计准则

包括：

-

CAD 双轨制

-
-

主流程不绑定求解器

-
-

STEP 统一几何接口

-
-

模板优先策略

-
-

Review Gate 审查机制

-

-

四层诊断系统

-
-

文件接口解耦

-

---

3 完整 Pipeline 架构

```
Input Adapter
↓
Intent / Geometry Spec
↓
CaseSpec
↓
Template Match
↓
Analysis Model
↓
CAD
↓
Mesh
↓
Solver Adapter
↓
Solver
↓
Postprocess
↓
Review Gate
↓
Library Update
```

---

4 每个模块详细职责

包含：

Input Adapter

输入来源识别

Intent / Geometry Spec

任务结构定义

CaseSpec

算例描述标准化

Template Match

模板匹配

Analysis Model

统一分析模型

CAD

CadQuery 自动建模 + 外部 CAD STEP

Mesh

Gmsh 网格生成

Solver Adapter

生成：

CalculiX .inp
Abaqus .inp
Nastran .bdf

Solver

执行求解

Postprocess

生成：

result_summary.json
field_manifest.json
history_data.csv

artifacts/
diagnostics.json

Review Gate

审查是否入库

Library Update

更新：

CaseSpec Library
Template Library
Solver Profiles

5　后处理结构（你认可的版本）

result_summary.json
field_manifest.json
history_data.csv
artifacts/
diagnostics.json

6　四层诊断系统

输入校验
接口校验
运行诊断
修复建议

如果你愿意，我可以在下一步帮你做一件 **非常关键的事情**：

我可以把这份 **pipeline 再升级成工业级架构图 + 数据接口规范**，包括：

- `case_spec schema`
-
-
- `analysis_model schema`
-

-

solver_adapter schema

-
-

result schema

-
-

review schema

-

这一步其实会直接决定：

你的 Auto CAD/CAE 系统未来能不能稳定扩展到 ToB。

ChatGPT 也可能会犯错。请核查重要信息。查看 Cookie 首选项。